

# A Post-Implementation Performance Prediction Method with HLS Optimization Directives

Jingyu Zhu, Yan Ding\*, Lu Xiao, Kenli Li, Chubo Liu, and Zheng Xiao\*  
College of Information Science and Engineering, Hunan University, China  
{zhujingyu, ding, dalu, lkl, liuchubo, zxiao}@hnu.edu.cn

**Abstract**—High-Level Synthesis (HLS) offers various optimization directives that enable designers to flexibly adjust hardware microarchitecture. However, existing HLS performance prediction methods typically rely on control data flow graphs generated (CDFG) from original HLS C/C++, which struggle to capture the complex interactions between directives and the resulting hardware resource reuse issues. To address these issues, this paper proposes a post-implementation performance prediction method tailored for directive-optimized circuits design, which utilizes a Graph Builder to integrate directive optimization and resource reuse information into the graph representation. In addition, the performance prediction model, which integrates a TransformerConv-based graph neural network (GNN) and an aggregation pool module, effectively captures key features related to post-implementation performance. Experimental results show that our method can reduce the prediction error of critical path delay (CP), power, and resource utilization to 3.87%~8.08%, significantly outperforming existing state-of-the-art methods. It also demonstrates excellent generalization on unseen kernels, providing a more effective and accurate performance prediction tool for HLS.

**Index Terms**—High-Level Synthesis, Directive-Optimized, Resource Reuse, Performance Prediction Model.

## I. INTRODUCTION

With the rapid development of 5G, autonomous driving, and large model technologies, hardware accelerator are playing a crucial role in enhancing computational performance and efficiency [1]. However, hardware programming typically requires extensive hardware knowledge and expert experience, along with significant effort to write Register-transfer-level(RTL) code, making the process both time-consuming and error-prone. To improve hardware development efficiency, High-Level Synthesis (HLS) technology has emerged as a solution.

HLS enables designers to create RTL hardware designs at a higher level of abstraction using programming languages such as C/C++, and to optimize micro-architecture configurations through various optimization directives [2]. However, the interactions between optimization directives are complex, and simply increasing the number of directives does not necessarily lead to better design outcomes. As a result, designers must carefully balance and select the optimal combination of directives from a vast design space, which introduces new challenges to design optimization. **First**, as the number of optimization directives increases, the design space expands exponentially, making it increasingly difficult to find the optimal design. **Second**, using HLS tools to evaluate the

performance of hardware implementations is highly time-consuming, as the process involves layout, routing, and other post-implementation steps. This time-consuming evaluation process further slows down the progress of design space exploration. Hence, the introduction of a fast and accurate performance prediction proxy model as a substitute for HLS tools becomes crucial.

Existing performance prediction models can be categorized into analytical models and learning-based models. Analysis models such as Lin-analyzer [3] and Comba [4] rely on extensive hardware knowledge and an understanding of regularized data flow designs. However, traditional analysis models often require substantial time to obtain necessary parameters, such as instruction latency, and fails to capture the heuristic optimization rules within HLS tools.

To address these limitations, researchers in [5] proposed a learning-based model for performance prediction. Recent work [6] has further advanced this direction by using control data flow graphs (CDFG) in combination with GNNs to predict design quality. GNN-DS [7] and HARP [8] directly insert directive nodes into the graph representation to improve performance prediction. However, this coarse-grained insertion approach has limitations: when applying the unroll directive, it may treat all expanded operations as a single node, making it challenging to capture the fine-grained impacts on execution order and resource utilization. In contrast, they [9] directly incorporates directive effects within a single for loop and uses a hierarchical GNN network to predict overall quality.

Nevertheless, existing methods, whether analysis or learning models, have yet to fully capture the complex effects of directive interactions and resulting resource reuse. Specifically, when *unroll* and *array partition* directives are applied together, parallelism and memory bandwidth are expected to improve. However, when multiple operations share the same hardware resource, this reuse can alter the intended scheduling order, impacting overall performance. Therefore, effectively incorporating directive interaction effects and resource reuse information into the graph representation, a more accurate reflection of operation scheduling and resource allocation in circuit design can be achieved, providing GNN models with richer design information and significantly enhancing the accuracy and reliability of performance prediction.

To address the above issues, this paper proposes a post-implementation performance prediction method tailored for directive-optimized circuit design. This approach not only

\*Corresponding author.

achieves precise performance prediction but also demonstrates excellent generalization capability. Our main contributions are as follows:

- 1) We propose a Graph Builder that constructs a directive-optimized graph by consider the effects of directives and hardware resource reuse, aligning more closely with the actual operation scheduling and resource allocation in hardware circuits. To our knowledge, this is the first approach in HLS performance prediction that introduces hardware resource reuse to simulate circuit characteristics, enabling more accurate performance prediction.
- 2) Based on the constructed directive-optimized graph, we propose a post-implementation performance prediction model that fully considers the impact of edge features on node information. This model integrates Transformer-Conv and aggregation pool modules to capture graph-level features related to performance metrics, while incorporating global information to enhance prediction accuracy and generalization capability.
- 3) Experimental results demonstrate that the proposed method outperforms state-of-the-art models across five performance prediction metrics; Specifically, the MAPE for LUT, FF, CP, and Power, and the MAE for DSP, are reduced to 8.08%, 3.87%, 4.34%, 6.49%, and 0.11%, respectively. Furthermore, it exhibits strong generalization capabilities on unseen kernels. Ablation studies further indicate that incorporating hardware resource reuse dependencies reduces prediction error.

The remainder of this paper is organized as follows: Section 2 introduces some preliminary concepts. Section 3 details the proposed method, including the directive-optimized graph construction, node and edge feature encoding, and the prediction model. Section 4 presents an extensive experimental analysis. Section 5 presents the conclusions.

## II. PRELIMINARIES

This section introduces the core foundational knowledge involved in this study, focusing on how programs are transformed into graph representations and the role of directives in HLS hardware design optimization. Additionally, we will discuss the principles of Graph Neural Networks and their application in hardware performance prediction. These concepts form the basis for the subsequent our approach and experiments.

### A. Program-to-Graph and Directives

In HLS tools, such as Vitis HLS, the source program is first compiled by an LLVM-based [10] front-end to generate an intermediate representation (IR), and then a Control Data Flow Graph (CDFG) is constructed based on the relationships between nodes and blocks. This graph clearly represents the dependencies and execution order of various operations within the program. The graph structure based on LLVM IR makes the transformation from high-level languages to hardware

designs more efficient, enabling designers to intuitively understand the execution logic of the program and analyze its hardware implementation details.

Furthermore, hardware design optimization involves the application of optimization directives (e.g., unroll, pipeline, array partition), which directly impact resource allocation and scheduling. To accurately capture the effects of these directives, they must be integrated into the graph structure to reflect actual resource allocation and scheduling. The directives involved in this paper are shown in Table I.

TABLE I: Configuration of directives.

| Pragma                    | Configuration         |
|---------------------------|-----------------------|
| Loop unrolling            | Unrolling factors     |
| Loop pipelining           | Enabled/Disabled      |
| Loop flattening           | Yes/No                |
| Array partitioning-type   | Block/Cyclic/Complete |
| Array partitioning-factor | Partitioning factors  |
| Function inlining         | Yes/No                |

### B. Graph Neural Networks

Graph Neural Networks [11] are a class of deep learning models specifically designed to process graph-structured data. By iterative propagation and aggregation of node information, GNNs learn feature representations of nodes and edges within the graph. Unlike traditional neural networks, GNNs can capture the topological structure of the graph and complex dependencies between nodes, making them well-suited for graph-related tasks. In the context of HLS, where the CDFG is generated from the intermediate representation, GNNs can effectively learn the dependencies between control flow and data flow, capturing the interactions between operations. Additionally, GNNs exhibit strong generalization capabilities, maintaining high accuracy in performance prediction across various design choices and optimization strategies. This is crucial for Design Space Exploration (DSE), as GNNs enable designers to quickly evaluate the performance of a large number of design points, accelerating the iterative optimization process and providing robust support for hardware microarchitecture optimization.

In summary, the application of GNNs can address the challenges of optimization directives in complex hardware designs, enhancing the accuracy and efficiency of design quality prediction and offering advanced solutions for hardware design optimization.

## III. METHODOLOGY

Fig. 1 illustrates the overall framework of our proposed post-implementation performance prediction method, which is divided into three stages: dataset generation, model training, and inference.

In the dataset generation stage, we randomly sample candidate optimization directives to determine the specific optimization strategy applied to the designs. Based on the IR generated by the HLS front end, the Graph Builder constructs a directive-optimized graph that incorporates resource reuse and directive

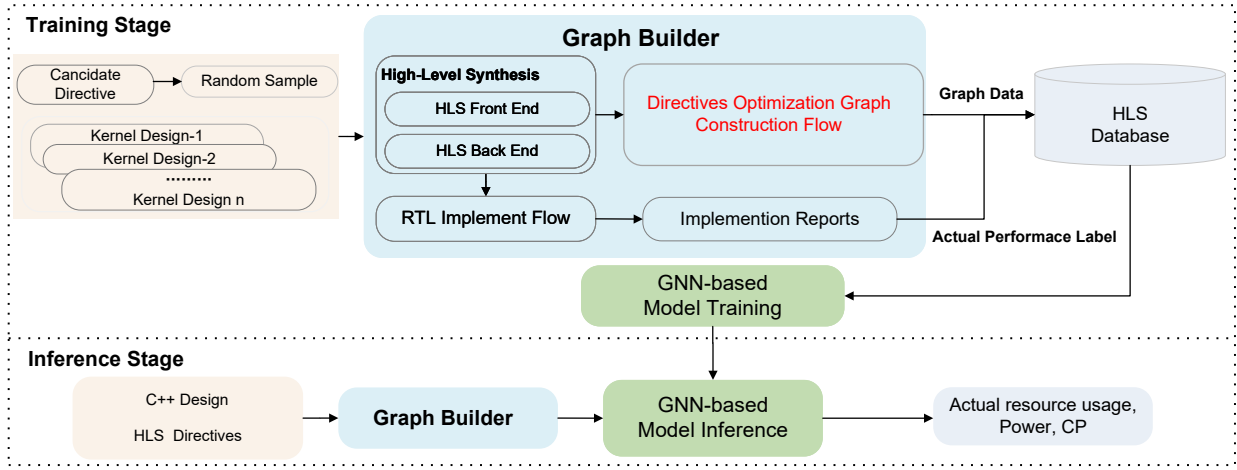


Fig. 1: Framework overview of the proposed method.

optimization effects, providing a more accurate simulation of resource allocation and operation scheduling in hardware circuits. This setup facilitates the use of GNN's powerful graph information learning capabilities for performance prediction. Subsequently, the C-to-bitstream design flow is completed to extract post-implementation performance metrics (LUT, FF, CP, Power, DSP). Using this information, we generate a directive-optimized graph representation corresponding to the optimization directives, along with true performance labels, to build a HLS performance prediction database.

During the training stage, we use the generated HLS database to train a model that integrates GNN and aggregation pool modules. Finally, in the inference stage, for a given test design, the directive-optimized graph representation generated by the Graph Builder is input into the trained model, enabling the prediction of post-implementation performance results without complete the post-implementation process.

#### A. Directives Optimization Graph Construction Flow

We use the Graph Builder to construct a graph representation of the design, which simulates the actual behavior of the hardware circuit. This graph representation incorporates control data flow information, the effects of various optimization directives, and hardware resource reuse details. Fig 2 illustrates our graph construction flow.

##### 1) Explicit Representation of Directive Optimization

The CDFG generated from LLVM IR (as shown in Fig.2(a)) provides a clear representation of the program's control structures and data dependencies, aiding designers in analyzing program behavior. However, optimization directives provided by HLS tools can alter the microarchitecture of the hardware circuit design. Specifically, applying the unroll directive duplicates the execution logic within loops to enhance parallelism, while the array partition directive increases memory bandwidth to improve throughput, thereby impacting the overall performance. Consequently, the interactions between directives can significantly change the data

flow execution patterns of the program, making accurate performance prediction essential.

To intuitively reflect the impact of directives on hardware design, we constructed a graph representation based on the frontend IR generated by HLS tools and the design principles of optimization directives. This graph representation effectively illustrates the effects of optimization directives on the source program, providing a clear graph structure for GNN-based performance prediction and facilitating the learning of directive interaction features (as shown in Fig.2(b)).

##### 2) Impact of hardware resource reuse

The optimized data flow graph effectively captures the execution characteristics of the program but often overlooks the impact of resource reuse caused by directive interactions. To more accurately model the resource allocation and operation scheduling in hardware circuits, we extend the optimization graph representation constructed in step (1) by incorporating resource dependency information derived from the backend of HLS tools. This enhancement improves the reliability of post-implementation performance prediction (as shown in Fig.2(c)).

More specifically, in Fig.2(c), the application of the unroll and array partition directives duplicates the execution logic and partitions array  $B$  into two segments, thereby improving parallelism and memory bandwidth. However, due to hardware resource reuse constraints, the two multiplication operations (mul1 and mul2) must share a single DSP resource. Consequently, mul2 must wait for mul1 to release the multiplier resource (e.g., DSP) before execution. To address this, a resource reuse dependency is introduced between these two multiplication operations, ensuring that mul1 and mul2 share the same DSP resource and that mul2 is scheduled to execute after mul1. This optimized graph representation more accurately reflects the resource allocation and operation scheduling in hardware circuits, enabling a more realistic simulation of hardware execution behavior.

```

void test(int A[16][16], int B[16][16],
         int C[16][16]) {
    #pragma HLS ARRAY_PARTITION
    variable=B dim=1 factor=2 type=cyclic
    int i, j;

    label0: for (i = 0; i < 16; i++) {
        label1: for (j = 0; j < 16; j++) {
            #pragma HLS UNROLL factor=2
            C[i][j] = A[i][j] * B[i][j];
        }
    }
}

```

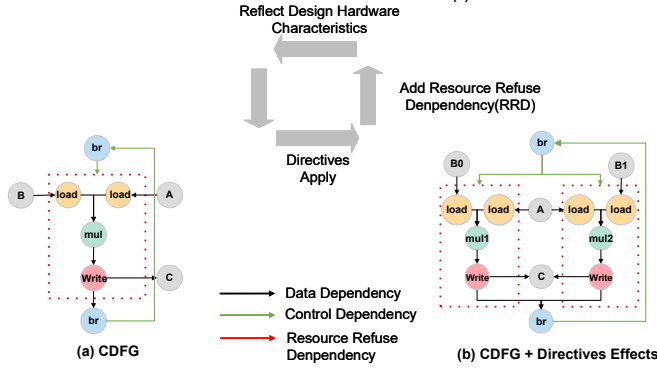


Fig. 2: Directive-optimization Graph Representation.

### B. Graph Feature Extraction and Annotation

The extraction of node and edge features from the directives optimization graph constructed by Graph Builder is crucial for model training (see Table II).

Firstly, node features are derived from the front-end of the HLS tool, extracting key information related to performance prediction, including node type, opcode type, latency, and resource usage (such as LUT, DSP, FF, etc.). Additionally, each node includes a "Reuse" flag, indicating whether the node reuses hardware resources. Node type and opcode type are encoded using one-hot vectors, while other features (such as latency and resource usage) are represented using numerical values. These features assist the GNN model in understanding the function and resource requirements of each node.

Edge features primarily describe the dependencies between nodes, including data dependencies, control dependencies, and resource reuse dependencies. In particular, resource reuse dependencies describe the operation scheduling and resource sharing in circuit design, aligning more closely with the characteristics of actual hardware execution. Comprehensive feature extraction and precise encoding enable the GNN model to gain deeper insights into the details of hardware design, helping it more effectively analyze and predict the performance of designs optimized by directives.

TABLE II: Node and edge features.

| Category      | Description | Values                              |
|---------------|-------------|-------------------------------------|
| Node Features | Type        | Instruction, Variable, etc.         |
|               | Opcode Type | Load, Mul, etc.                     |
|               | Delay       | Float values                        |
|               | LUT         | Integer values                      |
|               | DSP         | Integer values                      |
|               | FF          | Integer values                      |
|               | Reuse       | 0, 1                                |
| Edge Feature  | Type        | Control, Data, Call, Resource Reuse |

### C. GNN-Based Prediction Approach

Mainstream GNN models (such as GAT [12] and GraphSAGE [13]) typically focus on aggregating node features while often overlooking edge feature learning. However, in the directive-optimized graph constructed by Graph Builder, edge relationships contain rich circuit characteristics, including hardware resource reuse and the interactions between directive. To address this limitation, our performance prediction model (as shown in Fig.3) employs TransformerConv [14] as a foundational module for node information aggregation. TransformerConv introduces a self-attention mechanism similar to that in Transformers, enabling dynamic fusion of edge features between nodes, which allows for more effective handling of the graph data generated by Graph Builder. While learning node features, TransformerConv can also fully capture data dependencies, directive interactions, and resource reuse, thus extracting critical features related to performance metrics.

The learning of node features is illustrated by Equation (1),

$$\mathbf{x}'_i = \mathbf{W}_1 \mathbf{x}_i + \sum_{j \in \mathcal{N}(i)} \alpha_{i,j} (\mathbf{W}_2 \mathbf{x}_j + \mathbf{W}_5 \mathbf{e}_{ij}) \quad (1)$$

where the attention coefficient  $\alpha_{i,j}$  is calculated as follows:

$$\alpha_{i,j} = \text{softmax} \left( \frac{(\mathbf{W}_3 \mathbf{x}_i)^\top (\mathbf{W}_4 \mathbf{x}_j + \mathbf{W}_5 \mathbf{e}_{ij})}{\sqrt{d}} \right) \quad (2)$$

where  $\mathbf{W}_1, \mathbf{W}_2, \mathbf{W}_3, \mathbf{W}_4, \mathbf{W}_5$  are learnable weight parameters, and  $\mathbf{x}_i$  and  $\mathbf{e}_{ij}$  represent the feature embeddings of node  $i$  and the edge between nodes  $i$  and  $j$ , respectively. To further enhance the model's ability to learn graph-level features, we introduce an aggregation pool module [15] after three layers of TransformerConv to obtain a graph-level representation. Additionally, global information is incorporated into the graph-level representation, further enriching the information expression for circuit design and significantly improving the model's prediction accuracy and generalization capability. Finally, the graph-level and global features are concatenated and fed into a multi-layer perceptron (MLP) layer to predict performance metrics.

In summary, by integrating the TransformerConv model and aggregation pool, incorporating global information, we capture critical features related to performance prediction, thereby enhancing the performance prediction capability for circuit designs. Our prediction model framework is shown in Fig.3.

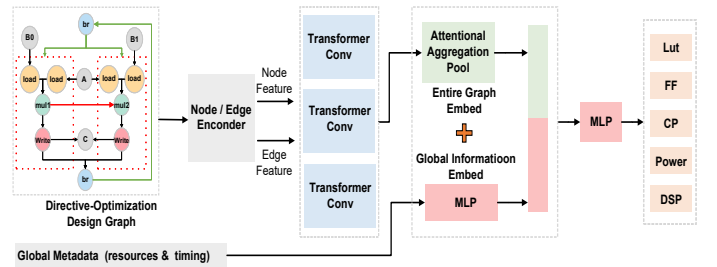


Fig. 3: The architecture of predictive model.

## IV. EXPERIMENT

### A. Experimental Setup

**Dataset.** We selected 19 representative kernel designs from the commonly used MachSuite [16] and PolyBench [17] benchmark suites. These kernels cover various types of computational complexity, including typical algorithms such as matrix multiplication, triangular solving, dynamic programming, and Jacobi iteration.

For each kernel design, we randomly generated 1,000 different directive configuration combinations, synthesized using Vitis HLS 2023.2 and implemented with Vivado to obtain actual hardware performance metrics, including resource usage (e.g., LUT, DSP, FF), critical path delay (CP), and power consumption. Finally, we combined each HLS kernel's generated directive-optimization graph representation with corresponding performance labels, constructing an HLS dataset to comprehensively evaluate the effectiveness of our method.

More specifically, these 19 kernels generate a total of 19,000 design points. Among them, design points from 15 kernels are used for training and testing, while the remaining 4 kernels are reserved to evaluate the model's generalization capability on unseen kernel designs. The specific details of the constructed HLS dataset are shown in Table III.

TABLE III: Statistics of databases which consist of 19 kernels.

| Version          | Kernels | Range      | LUT          | FF           | CP         | Power        | DSP       |
|------------------|---------|------------|--------------|--------------|------------|--------------|-----------|
| Vitis HLS 2023.2 | 19      | min<br>max | 19<br>138942 | 26<br>246368 | 1.48<br>10 | 0.24<br>2.97 | 0<br>1428 |

**Baseline.** To validate the effectiveness of our method, we compared it with four baseline methods: PNA [18], IronMan-Pro [6], GNN-DSE [7], and HGP [19] represent kernel characteristics through data flow graphs, while GNN-DSE directly inserts directive nodes into the graph representation. In our experiments, we primarily focus on prediction accuracy and generalization capability, using Mean Absolute Percentage Error (MAPE) to evaluate LUT, FF, CP, and Power metrics, and Mean Absolute Error (MAE) for the DSP metric. Additionally, we implemented the Graph Builder in Python to construct the directive-optimized graph representation and perform feature extraction and encoding. Both our method and the baseline GNN methods were implemented using Python and PyTorch Geometric.

### B. Model Evaluation

In the above experimental setup, we selected 15,000 design points (from 15 kernel designs), with 80% used for training and 20% for testing, to evaluate the performance of all models.

The performance of all models is shown in Fig. 4, the proposed model outperforms baseline models (PNA, IronMan-Pro, GNN-DSE, and HGP) across five evaluation metrics, achieving the lowest MAPE (MAE) values. Compared to PNA and IronMan-Pro, the proposed approach demonstrates significant improvements in all performance prediction metrics, primarily due to its effective capture of node features

and edge dependencies in the directive optimization graph. Although GNN-DSE and HGP reduce performance prediction errors to some extent, the proposed method still surpasses them, further demonstrating its ability to effectively learn hardware resource reuse, directive interaction information, and correlations between performance metrics.

Overall, our model reduces the prediction errors for LUT, FF, CP, and Power to 8.08%, 3.87%, 4.34%, and 6.49%, respectively. And achieves an MAE of 0.11 for DSP, which is significantly lower than that of other models, this low error indicates a high consistency between predicted and actual values, further validating the model's high accuracy and stability.

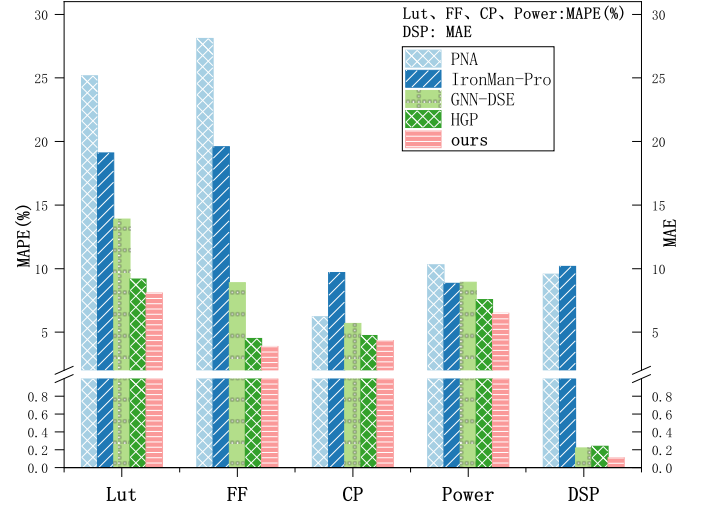


Fig. 4: Performance comparison.

### C. Effectiveness of Resource Reuse

In actual RTL circuits, resource reuse is closely related to operation scheduling and resource allocation, making it essential to consider resource reuse information for reducing model prediction errors. To assess the impact of resource reuse information, we compared the model's performance with (W/) and without (W/O) this information. As shown in Table IV, incorporating resource reuse information significantly reduces prediction errors across five metrics. Specifically, the MAPE for LUT, FF, CP, and Power decreased from 9.41%, 4.52%, 4.54%, and 7.25% to 8.08%, 3.87%, 4.34%, and 6.49%, respectively, while the MAE for DSP dropped from 0.20 to 0.11. These results indicate that incorporating resource reuse information enables the model to more accurately capture the operation scheduling and resource allocation characteristics of hardware circuits, effectively reducing prediction errors.

TABLE IV: Performance comparison of resource reuse.

| Method | Configuration      | MAPE (%) |      |      |       | MAE  |
|--------|--------------------|----------|------|------|-------|------|
|        |                    | Lut      | FF   | CP   | Power | DSP  |
| Ours   | W/O Resource Reuse | 9.41     | 4.52 | 4.54 | 7.25  | 0.20 |
|        | W/ Resource Reuse  | 8.08     | 3.87 | 4.34 | 6.49  | 0.11 |

TABLE V: Generalization performance comparison of different models on unseen Kernels.  
GNN-DSE is abbreviated as GD in the table.

| Kernels     | MAPE (%)     |              |              |              |              |              |             |             |             |             |             |             |              |              |              |              | MAE          |             |             |             |
|-------------|--------------|--------------|--------------|--------------|--------------|--------------|-------------|-------------|-------------|-------------|-------------|-------------|--------------|--------------|--------------|--------------|--------------|-------------|-------------|-------------|
|             | Lut          |              |              |              | FF           |              |             |             | CP          |             |             |             | Power        |              |              |              | DSP          |             |             |             |
|             | PNA          | GD           | HGP          | ours         | PNA          | GD           | HGP         | ours        | PNA         | GD          | HGP         | ours        | PNA          | GD           | HGP          | ours         | PNA          | GD          | HGP         | ours        |
| viterbi     | 71.95        | 46.67        | 18.76        | <b>18.33</b> | 57.5         | 20.38        | 9.06        | <b>6.91</b> | 10.74       | 16.38       | 9.01        | <b>6.51</b> | 28.96        | 16.94        | 37.03        | <b>8.97</b>  | 20.51        | 4.39        | 3.67        | <b>3.21</b> |
| trmm        | 49.59        | 23.23        | 22.94        | <b>17.96</b> | 39.69        | 10.21        | 12.26       | <b>8.87</b> | 12.62       | 7.11        | <b>6.92</b> | 8.89        | 13.71        | 10.23        | 15.51        | <b>10.21</b> | 12.89        | 0.31        | 3.17        | <b>0.09</b> |
| syrc        | 53.80        | 23.71        | 30.32        | <b>20.96</b> | 64.67        | 14.4         | 13.18       | <b>9.44</b> | 9.19        | 7.98        | <b>6.51</b> | 7.60        | 31.83        | 17.91        | 22.53        | <b>14.31</b> | 32.74        | 0.21        | 11.31       | <b>0.08</b> |
| 2mm         | 27.61        | 17.09        | 14.72        | <b>12.53</b> | 39.85        | 7.37         | 4.72        | <b>3.69</b> | 6.02        | 6.84        | 6.92        | <b>5.84</b> | 7.88         | 4.51         | 7.24         | <b>4.11</b>  | 6.36         | 0.09        | 1.65        | <b>0.02</b> |
| <b>Mean</b> | <b>50.74</b> | <b>27.68</b> | <b>21.69</b> | <b>17.45</b> | <b>45.68</b> | <b>13.09</b> | <b>9.81</b> | <b>7.23</b> | <b>9.64</b> | <b>9.57</b> | <b>7.34</b> | <b>7.21</b> | <b>20.60</b> | <b>12.40</b> | <b>20.58</b> | <b>9.40</b>  | <b>18.13</b> | <b>1.25</b> | <b>4.95</b> | <b>0.85</b> |

## D. Generalization on Unseen Kernel

To evaluate the model's generalization capability, we conducted experiments on four unseen kernel designs (viterbi, trmm, syrc, and 2mm). As shown in TableV, the proposed model outperforms other baseline models (PNA, GNN-DSE, and HGP) on unseen kernel designs, achieving lower MAPE (MAE) across five metrics. Specifically, the average MAPE of our model for LUT, FF, CP, and Power decreased to 17.45%, 7.23%, 7.21%, and 9.40%, respectively, while the MAE for DSP was significantly reduced to 0.85, much lower than other models. The experimental results validate the strong generalization capability of the proposed method on unseen kernel designs, further demonstrating the effectiveness of combining the directive optimization graph representation with the performance prediction model architecture. This approach enables a more comprehensive modeling of the commonalities and differences across various designs, highlighting its robustness and adaptability.

## V. CONCLUSION

In this paper, we propose a novel post-implementation performance prediction method for directive-optimized designs. This approach involves the development of a Graph Builder, which constructs a directive optimization graph that integrates both directive optimization effects and resource reuse information. By combining an ensemble GNN model with an aggregation module, we extract key features relevant to performance prediction, such as resource allocation and operation scheduling, to achieve precise performance estimation. Experimental results demonstrate that our method significantly outperforms the baseline approaches across five metrics and exhibits strong generalization ability on unseen kernels.

## ACKNOWLEDGEMENT

This work was supported in part by National Natural Science Foundation of China (Nos. 62472155, 62202154), the science and technology innovation Program of Hunan Province (No.2024RC3076), and the Fundamental Research Funds for the Central Universities.

## REFERENCES

[1] Biagio Peccerillo, Mirco Mannino, Andrea Mondelli, and Sandro Bartolini. A survey on hardware accelerators: Taxonomy, trends, challenges, and perspectives. *Journal of Systems Architecture*, 129:102561, 2022.

[2] *Vitis High-Level Synthesis User Guide (UG1399)*, 2022. Retrieved from Xilinx Documentation: <https://docs.xilinx.com/r/2022.1-English/ug1399-vitis-hls>.

[3] Guanwen Zhong, Alok Prakash, Yun Liang, Tulika Mitra, and Smail Niar. Lin-analyzer: A high-level performance analysis tool for fpga-based accelerators. In *DAC*, pages 1–6, 2016.

[4] Jieru Zhao, Liang Feng, Sharad Sinha, Wei Zhang, Yun Liang, and Bing-sheng He. Comba: A comprehensive model-based analysis framework for high level synthesis of real applications. In *ICCAD*, pages 430–437. IEEE, 2017.

[5] Steve Dai, Yuan Zhou, Hang Zhang, Ecenur Ustun, Evangeline FY Young, and Zhiru Zhang. Fast and accurate estimation of quality of results in high-level synthesis with machine learning. In *FCCM*, pages 129–132. IEEE, 2018.

[6] Nan Wu, Yuan Xie, and Cong Hao. Ironman: Gnn-assisted design space exploration in high-level synthesis via reinforcement learning. In *Proceedings of the 2021 on Great Lakes Symposium on VLSI*, pages 39–44, 2021.

[7] Atefeh Sohrabizadeh, Yunsheng Bai, Yizhou Sun, and Jason Cong. Automated accelerator optimization aided by graph neural networks. in 2022 59th acm. In *DAC*, 2022.

[8] Atefeh Sohrabizadeh, Yunsheng Bai, Yizhou Sun, and Jason Cong. Robust gnn-based representation learning for hls. In *ICCAD*, pages 1–9. IEEE, 2023.

[9] Mingzhe Gao, Jieru Zhao, Zhe Lin, and Minyi Guo. Hierarchical source-to-post-route qor prediction in high-level synthesis with gnns. In *DATE*, pages 1–6. IEEE, 2024.

[10] Chris Latner and Vikram Adve. Llvm: A compilation framework for lifelong program analysis & transformation. In *International symposium on code generation and optimization, 2004. CGO 2004.*, pages 75–86. IEEE, 2004.

[11] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S Yu Philip. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems*, 32(1):4–24, 2020.

[12] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.

[13] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.

[14] Yunsheng Shi, Zhengjie Huang, Shikun Feng, Hui Zhong, Wenjin Wang, and Yu Sun. Masked label prediction: Unified message passing model for semi-supervised classification. *arXiv preprint arXiv:2009.03509*, 2020.

[15] Yujia Li, Chenjie Gu, Thomas Dullien, Oriol Vinyals, and Pushmeet Kohli. Graph matching networks for learning the similarity of graph structured objects. In *ICML*, pages 3835–3845. PMLR, 2019.

[16] Brandon Reagen, Robert Adolf, Yakun Sophia Shao, Gu-Yeon Wei, and David Brooks. Machsuite: Benchmarks for accelerator design and customized architectures. In *ISWC*, pages 110–119. IEEE, 2014.

[17] T. Yuki and L.-N. Pouchet. PolyBench/C. [Online]. Available: <https://web.cse.ohio-state.edu/~pouchet.2/software/polybench/>.

[18] Nan Wu, Hang Yang, Yuan Xie, Pan Li, and Cong Hao. High-level synthesis performance prediction using gnns: Benchmarking, modeling, and advancing. In *DAC*, pages 49–54, 2022.

[19] Huizhen Kuang, Xianfeng Cao, Jingyuan Li, and Lingli Wang. Hgbo-dse: Hierarchical gnn and bayesian optimization based hls design space exploration. In *ICFPT*, pages 106–114. IEEE, 2023.